Agro Eye Project Report

Comprehensive implementation and integration documentation generated on 2026-03-02 17:09.

1) Project Overview

Agro Eye is a Flutter-based mobile application for plant leaf disease detection. The application allows users to authenticate, capture or select leaf images, run local on-device inference, and display disease/healthy predictions with confidence.

Core goals of the project:

- Build a practical mobile UX for disease scanning (camera + gallery).
- Integrate a deep learning model with low-latency on-device inference.
- Provide robust preprocessing and readable prediction output.
- Keep the app functional even when model runtime issues occur (fallback-safe architecture).

2) Technology Stack

Application Layer

- Flutter (Dart SDK constraint: ^3.8.1)
- Firebase Core + Firebase Auth for app initialization and authentication
- image_picker for camera/gallery image input
- image package for image decoding, resizing, and pixel processing
- tflite_flutter for TensorFlow Lite runtime execution

ML & Conversion Layer

- PyTorch checkpoints (.pth) as model training artifacts
- ONNX as interchange format
- onnx2tf for ONNX → TensorFlow/TFLite conversion
- TensorFlow Lite model deployed as app asset

3) High-Level App Architecture

The model integration follows a service-oriented pattern where UI concerns and inference logic are separated. The UI triggers image acquisition and classification requests while a dedicated classifier service handles model loading, preprocessing, tensor execution, and probability postprocessing.

Main flow

- User opens Home screen and classifier initializes in the background.
- User captures/selects an image.
- Image is sent to PlantClassifierService.classifyImage(...).
- Service decodes + resizes image to 224×224 and normalizes with ImageNet stats.

- TFLite inference returns logits/probabilities for 8 classes.
- Top class + confidence are returned and rendered in the UI.

4) Files and Responsibilities

Key Flutter files

- lib/main.dart: Firebase initialization and app bootstrap
- lib/home_screen.dart: crop selection, gallery/camera triggers, and result display
- lib/services/plant_classifier_service.dart: TFLite lifecycle + preprocessing + inference
- pubspec.yaml: dependencies and model/labels asset registration

Platform configuration

- android/app/src/main/AndroidManifest.xml: camera/media permissions
- android/app/build.gradle.kts: noCompress("tflite") for model packaging
- ios/Runner/Info.plist: camera and photo-library usage descriptions

5) Model Evolution and Integration Journey

The integration journey included multiple iterations due to cross-platform runtime and conversion constraints. Initial attempts used ONNX Runtime directly in Flutter, but Android native library loading issues blocked reliable deployment. The project then moved to TensorFlow Lite for stable mobile inference packaging.

Important technical milestones

- Initial ONNX Runtime approach encountered Android shared-library loading issues.
- Migration to TFLite runtime in Flutter improved deployment reliability.
- Python compatibility issues (very recent Python versions) broke ML package installation.
- A compatible conversion environment (Python 3.11-based) was used for successful conversion.
- Input tensor format mismatch (NCHW vs NHWC) was corrected in Flutter preprocessing.
- Final integration switched active model to best_mobilenetv3_small checkpoint.

6) Final Integrated Model (Current State)

Current active mobile model

- Source checkpoint: output_comparison/best_mobilenetv3_small.pth
- Converted deployment model: assets/ml/plant_disease_model.tflite
- Model family: MobileNetV3 Small
- Checkpoint validation accuracy: 100.0 (as stored in checkpoint metadata)
- Epoch in checkpoint metadata: 15

Input/Output contract

- Input shape: [1, 224, 224, 3]
- Input dtype: float32
- Tensor format: NHWC (channels-last)
- Output shape: [1, 8]
- Output dtype: float32

Class labels

Index	Label
0	Apple Scab
1	Apple Black Rot
2	Apple Cedar Rust
3	Apple Healthy
4	Grape Black Rot
5	Grape Esca
6	Grape Leaf Blight

7	Grape Healthy

7) Inference Pipeline Details

Image preprocessing

- Decode image bytes into RGB pixel matrix.
- Resize image to 224×224.
- Convert each channel from [0,255] to [0,1].
- Normalize with ImageNet mean/std per channel.
- Build tensor in NHWC format: [batch, height, width, channels].

Post-processing

- Raw model output is transformed via softmax.
- Highest-probability label is selected as top prediction.
- Confidence is presented as percentage in UI.
- The label is also interpreted as Healthy vs Disease based on class name.

8) Conversion Pipeline Used for MobileNetV3-Small

Implemented script: convert_best_mobilenetv3_small.py

- Loads checkpoint and extracts state_dict + class_names.
- Rebuilds torchvision MobileNetV3-Small with 8-class classifier head.
- Exports model to ONNX (opset 13, dynamic batch axis).
- Converts ONNX to TensorFlow/TFLite using onnx2tf.
- Copies generated float32 TFLite model to assets/ml/plant_disease_model.tflite.
- Updates assets/ml/model_metadata.json for deployment traceability.

9) Platform and Packaging Considerations

Android

- Camera and media permissions are declared in AndroidManifest.xml.
- TFLite files are excluded from compression via aaptOptions.noCompress('tflite').
- This avoids model extraction/reading issues at runtime.

iOS

- Info.plist contains NSCameraUsageDescription and NSPhotoLibraryUsageDescription.
- These are required for camera/gallery workflows on iOS.

10) Known Issues and Resolutions

Issue: Python package incompatibility with bleeding-edge Python

- Symptom: TensorFlow/ONNX-related packages failing to install.
- Resolution: Use a compatible Python environment for model conversion (e.g., 3.11).

Issue: ONNX Runtime Android native loading failure

- Symptom: Native shared library errors on Android runtime.
- Resolution: Move inference runtime to TensorFlow Lite in Flutter.

Issue: Bad state / failed precondition at inference time

- Symptom: Runtime error during _interpreter.run(...).
- Root cause: Input tensor channel ordering mismatch.
- Resolution: Changed preprocessing to NHWC [1,224,224,3].

11) Operational Runbook

Day-to-day app run

- flutter pub get
- flutter run -d emulator-5554 (or any connected device)

Regenerating model from best_mobilenetv3_small

- Ensure Python conversion environment has torch, torchvision, onnx2tf and dependencies.
- Run: python convert_best_mobilenetv3_small.py
- Rebuild Flutter app so updated asset is packaged.

12) Recommendations for Next Iteration

- Add confidence thresholding and uncertain/abstain class handling.
- Add model version display in-app (read from model_metadata.json).
- Persist inference history with image, label, confidence, timestamp.
- Add instrumentation logs for interpreter input/output signatures on startup.
- Create a small automated validation script that compares ONNX vs TFLite outputs on sample images.

13) Summary

Agro Eye now runs a fully integrated, on-device MobileNetV3-Small disease classifier via TFLite. The app pipeline is stable, model assets are properly packaged, preprocessing is aligned with the deployed tensor contract, and the conversion workflow is reproducible through the project scripts.